

## 2. Bases de la Programación:

### 2.1. ¿Qué es un lenguaje de programación?

*“Un lenguaje de programación es un lenguaje formal (o artificial, es decir, un lenguaje con reglas gramaticales bien definidas) que le proporciona a una persona, en este caso el programador, la capacidad de escribir (o programar) una serie de instrucciones o secuencias de órdenes en forma de algoritmos con el fin de controlar el comportamiento físico o lógico de un sistema informático, de manera que se puedan obtener diversas clases de datos o ejecutar determinadas tareas”*

El lenguaje de programación de Arduino está basado en C, el cual está dentro de los más populares por su eficiencia y control. Sin embargo, es más complejo que otros como Python.

### 2.2. Estructura General

Primero hay que entender los 3 entornos básicos en los que se trabaja:

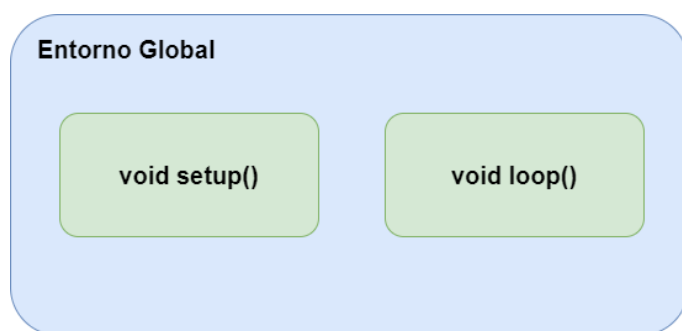


Figura 13: Diagrama del entorno de programación Arduino.

#### 2.2.1. Entorno Global

Esta es la zona de código que se encuentra afuera de los voids. Aquí se suelen incorporar las librerías, definiciones y variables de tipo global (más adelante se profundiza). Básicamente aquí va todo lo que el código va a necesitar acceder desde diferentes entornos locales.

#### 2.2.2. void setup()

El *void setup()* es lo que se conoce como un entorno local, y este es el primero que se ejecuta en el Arduino. Tal como dice su nombre (*setup = configuración*), es donde se realiza las configuraciones necesarias para trabajar y sólo se ejecuta 1 vez.

#### 2.2.3. void loop()

Este es el segundo entorno local base de nuestro código y es donde lo principal se va a ejecutar. Tal como dice su nombre (*loop = bucle*) este entorno se ejecuta indefinidamente una y otra vez.

## 2.3. Variables

Las variables son la unidad base de información de nuestro código. En ellas se pueden guardar todo tipo de datos para poder manipularlas, tomar decisiones, etc.

### 2.3.1. Variables globales

Las variables globales son las creadas en el entorno global del código, estas variables pueden ser accedidas desde cualquier entorno local y su valor va a ser el mismo. Si un entorno local la modifica, el resto de entornos va a ver este cambio. Es importante que estas variables sean únicas, es decir, cuando se crean, no se puede crear otra con el mismo nombre dentro de ningún entorno.

### 2.3.2. Variables locales

Las variables locales son las que se crean dentro de los entornos locales. Estas variables sólo existen dentro del entorno donde se crearon y no pueden ser utilizadas en uno diferente. 2 entornos locales diferentes pueden tener una variable con el mismo nombre, la información que tenga una puede ser diferente de la otra y si se cambia, no va a afectar al resto.

### 2.3.3. Tipos de variables

Para crear las variables, se tiene que definir el tipo de información que va a ir dentro, si luego se intenta guardar algo de diferente tipo, se genera un error. En la siguiente tabla se ven los tipos de variables que más vamos a utilizar:

| Tipo   | Descripción                         | Ejemplo                        |
|--------|-------------------------------------|--------------------------------|
| int    | Número entero                       | int numero = 7;                |
| float  | Número decimal                      | float decimal = 3.14;          |
| char   | Carácter                            | char letra = 'a';              |
| String | Cadena de caracteres (palabras)     | String palabra = "Hola mundo"; |
| bool   | Valor de verdad (verdadero o falso) | bool flag = true;              |

Tabla 2: Tipos de variables

**¿Sabías qué?** Existen muchos más tipos de variables, las cuales los puedes encontrar en la documentación de Arduino junto a muchas cosas más.

[www.arduino.cc/reference/es/](http://www.arduino.cc/reference/es/)

### 2.3.4. Arreglos (listas)

Hasta ahora hemos visto que las variables nos permiten guardar información, sin embargo, sólo podemos guardar un elemento por variable. Para abordar esto, existe algo llamado arreglos, o también conocidos como listas, que permiten guardar varios elementos del mismo tipo en la misma variable.

Para crearlas, es necesario definir el tipo, el nombre, y el tamaño (entre corchetes []):

```
1 int lista_de_numeros[5];
2 int otra_lista_de_numeros[]={1,2,3};
3 float lista_de_decimales[10];
4 char lista_de_letras[] = {"h","o","l","a"};
```

Notar que se pueden crear arreglos vacíos o ya con datos dentro.

La forma en cómo están ordenados los elementos dentro de un arreglo es comenzando desde el 0, es decir, para una lista con 5 elementos, estos van a estar numerados del 0 al 4:

| lista_de_numeros |   |   |   |   |   |
|------------------|---|---|---|---|---|
| Índice           | 0 | 1 | 2 | 3 | 4 |
| Valor            | 3 | 1 | 4 | 1 | 6 |

Ahora veamos un ejemplo básico de cómo se podrían utilizar los arreglos:

Primero creamos una lista de tamaño 2:

```
1 int lista[2];
```

Luego le asignamos valores a cada elemento:

```
1 lista[0] = 3;
2 lista[1] = 2;
```

Ahora con la lista llena de valores, se pueden realizar distintas operaciones, por ejemplo, una suma simple:

```
1 int resultado = lista[0] + lista[1];
```

En este caso, el valor guardado en resultado va a ser igual a 5 (2 + 3).

## 2.4. Lógica proposicional

La lógica proposicional hace referencia a aquellas afirmaciones que se pueden definir como verdadero o falso (“¿Esto es igual a esto otro?”, “¿Esto es más grande que esto otro?, etc”) y cómo interactúan unas con otras.

Dentro de Arduino tenemos 2 comparadores lógicos, el && y el ||.

### 2.4.1. Comparador lógico &&

Este comparador significa “y”. Se utiliza en preguntas del tipo: “Esta afirmación y esta otra afirmación”. Las combinaciones posibles se muestran a continuación:

| Afirmación A | Afirmación B | A && B    |
|--------------|--------------|-----------|
| Verdadero    | Verdadero    | Verdadero |
| Verdadero    | Falso        | Falso     |
| Falso        | Verdadero    | Falso     |
| Falso        | Falso        | Falso     |

Tabla 3: Tabla de verdad para el Y

### 2.4.2. Comparador lógico ||

Este comparador significa “o”. Se utiliza en preguntas del tipo: “Esta afirmación o esta otra afirmación”. Las combinaciones posibles se muestran a continuación:

| Afirmación A | Afirmación B | A    B    |
|--------------|--------------|-----------|
| Verdadero    | Verdadero    | Verdadero |
| Verdadero    | Falso        | Verdadero |
| Falso        | Verdadero    | Verdadero |
| Falso        | Falso        | Falso     |

Tabla 4: Tabla de verdad para el O

## 2.5. Aritmética

### 2.5.1. Comparación Aritmética

Cuando estamos trabajando con variables de tipo numéricas, podemos realizar comparaciones entre 2 variables y así poder crear afirmaciones. Las diferentes formas de comparación se pueden ver a continuación:

| Nombre         | Símbolo |
|----------------|---------|
| Igual          | A == B  |
| Desigual       | A != B  |
| Menor estricto | A < B   |
| Menor o Igual  | A <= B  |
| Mayor estricto | A > B   |
| Mayor o Igual  | A >= B  |

Tabla 5: Comparadores aritméticos

### 2.5.2. Operadores aritméticos

Por último, como estamos trabajando con números, podemos realizar operaciones matemáticas con ellos como sumar, restar, multiplicar y dividir(+, -, \*, /).

```

1  int a = 1;
2  int b = 5;
3  int c;
4  c = a + b;

```

En este ejemplo, el valor de la variable `c` va a ser 6. Para evitar algún tipo de problema, evita sumar variables de distinto tipo (Por ejemplo: `int + float`).

## 2.6. Consola Serial

Para poder visualizar mejor los contenidos siguientes, vamos a ver una pequeña introducción a la consola serial, más adelante en la Unidad 4 lo profundizaremos.

Con el siguiente código y mediante el `Serial.println()` vamos a poder mostrar cosas por la consola serial de Arduino:

```

1  int variable = 1;
2
3  void setup(){
4      Serial.begin(9600);
5  }
6
7  void loop(){
8      Serial.println("Una frase");
9      Serial.println(variable);
10 }

```

## 2.7. Estructuras condicionales

Las estructuras condicionales son las herramientas que tenemos para realizar diferentes acciones y controlar el flujo de nuestro código.

Estas funcionan en base a nuestras variables y básicamente son preguntas que podemos realizar dentro del código.

### 2.7.1. if

La estructura más básica de todas es el `if` que significa “Si esto es verdad”.

La estructura consta de un `if`, una condición entre paréntesis, y lo que se quiere hacer en caso de que la condición sea verdadera entre llaves:

```

1  if(condición){
2      //hacer algo
3  }

```

La condición debe ser algo que pueda ser verdadero o falso, la más típica a utilizar es la de verificar si una variable es igual a algo, lo cual se logra con el símbolo “==”:

```

1  numero == 3 //se pregunta si el número es
    igual 3

```

También se pueden encadenar muchos `if` con la estructura `if-else if` de la siguiente manera:

```

1  int numero = 2;
2  if(numero == 1){
3      //este pedazo de código se va a ignorar
        porque la condición es falsa
4  }else if(numero == 2){
5      //este código si se va a ejecutar porque
        la condición es verdadera
6  }else if(numero == 3){
7      //esta condición no se alcanza a verificar
        porque ya se encontró la verdadera
8  }

```

Por último, esto se puede complementar con la estructura `if-else if-else`, donde el último `else` está para considerar todo el resto de casos:

```

1  int numero = 2;
2  if(numero == 1){
3      //este pedazo de código se va a ignorar
        porque la condición es falsa
4  }else if(numero == 3){
5      //este pedazo de código se va a ignorar
        porque la condición es falsa
6  }else{
7      //este código es el que SI se ejecuta
8  }

```

Ahora en TinkerCad, prueba el siguiente código y varía las variables `a` y `b`, para modificar la salida (también prueba añadiendo un `if-else`):

```

1  int a = 10;
2  int b = 5;
3
4  void setup(){
5      Serial.begin(9600);
6  }
7
8  void loop(){
9      if (a<b){
10         Serial.println("A es mayor que B");
11     }else{
12         Serial.println("El if es FALSO");
13     }
14     while(true);
15 }

```

### WARNING:

- El `if` puede ir solo.
- Se pueden tener tantos `else if` como se quieran, pero siempre deben ir acompañados de un `if` al principio
- Sólo se puede tener un único `else`, y puede ir inmediatamente después de un `if` o un `else if`

### 2.7.2. while

El *while* es muy parecido al *if*, sólo que significa “Mientras la condición sea verdadera”, o sea, que va a ejecutar indefinidamente hasta que la condición se vuelva falsa.

Es importante fijarse que al momento de establecer la condición, esta en algún momento se vuelva falsa, ya que si no, el código se va a quedar atrapado por siempre dentro del **while**.

```
1  int contador = 0;
2  while(contador<5){
3      //en este caso el código se quedará
        ejecutando eternamente ya que contador
        no cambia.
4  }
```

```
1  int contador = 0;
2  while(contador<5){
3      contador = contador + 1;
4      //en este caso, el código se ejecutará 5
        veces y luego saldrá del while
5  }
```

Ahora para practicar, prueba el siguiente código en TinkerCad y juega con él:

```
1  int a = 0;
2
3  void setup(){
4      Serial.begin(9600);
5  }
6
7  void loop(){
8      while (a<10){
9          Serial.println("Estoy en el While");
10         Serial.println(a);
11         a += 2;
12     }
13     while(true);
14 }
```

### 2.7.3. for

La última estructura que vamos a ver es bastante compleja, pero cuando se utiliza bien es bastante poderosa, el *for*.

A diferencia del **if** o el **while**, dentro de los paréntesis no va únicamente una condición, si no que lo siguiente: variable, condición, modificador:

```
1  for(variable;condición;modificador){
2      //Código del for
3  }
```

En el código anterior, primero se declara una variable, la cual usualmente es llamada *i*, inicializada en 0:

```
1  for(int i = 0;condición;modificador){
2      //Código del for
3  }
```

Luego se declara la condición, la cual debe contener a la variable anteriormente creada (*i*):

```
1  for(int i = 0; i < 3;modificador){
2      //Código del for
3  }
```

Y por último el modificador, el cual realiza una operación sobre la variable (*i*) en cada iteración del *for*. Este modificador puede ser cualquier operación matemática que se quiera, con la precaución de que en algún momento la condición se hará falsa (como en el **while**):

```
1  for(int i = 0; i < 3;i++){
2      //Lo que está dentro del for se ejecutará
        3 veces
3  }
```

En este caso se utiliza *i++*, lo cual significa que en cada iteración se sumará 1 a la variable, también existe *i--*, que resta, o se puede usar cualquier otra operación matemática de la forma *i = <operación>*.

Ahora para practicar el *for* y reforzar el concepto de listas, ejecuta el siguiente código en TinkerCad. Prueba jugar con listas de diferente tipo.

```
1  int lista[] = {3,1,4,1,5};
2  int largo_lista = 5;
3
4  void setup(){
5      Serial.begin(9600);
6  }
7
8  void loop(){
9      for(int i = 0; i < largo_lista;i++){
10         Serial.println("Mostrar item lista");
11         Serial.println(lista[i]);
12     }
13     while(true);
14 }
```

## 2.8. Funciones

Como vimos antes, existen 2 entornos locales nativos en Arduino, pero nosotros podemos crear nuestros propios entornos para hacer nuestro código más ordenado.

Estos entornos propios se denominan *Funciones*, las cuales deben ser definidas dentro del entorno global, es decir, no pueden ser creadas dentro del **void setup()** o el **void loop()**.

De esta forma podemos escribir códigos complejos de tareas que se hagan repetidamente, y así sólo usamos la función en vez de escribir mucho código cada vez que necesitemos hacer algo.

### 2.8.1. Funciones básicas

La función más básica que se puede crear es de tipo void, lo que significa que no nos va a devolver ningún valor, sólo realiza una acción:

```
1 void mi_funcion(){
2     //aquí va el código de la acción que se
3     quiere hacer
}
```

Y luego cuando se quiera utilizar esta función, se debe llamar en el **void loop()** o en el **void setup()** de la siguiente manera:

```
1 void loop(){
2     mi_funcion();
3 }
```

Desde esta función se podrá acceder sólo a las variables globales y las que se creen dentro de la misma, pero no a las creadas en el **void loop()** o en el **void setup()**.

### 2.8.2. Funciones con entrada(Input)

Cuando necesitemos pasar información a nuestra función desde el entorno del que se llama, se pueden establecer lo que se conoce como inputs.

Con esto le decimos a la función de que tiene que recibir un dato, el cuál tiene que ser de un tipo específico que le indiquemos dentro de los paréntesis junto a un nombre para utilizarlo dentro de la función:

```
1 void mi_funcion_input(int a, char b){
2     //aquí va el código de la acción que se
3     quiere hacer
4     //se pueden utilizar las variables a y b
}
```

Notar que si se quieren usar más de un input (pueden ser todos los que quieran), se deben separar con una coma. Luego cuando se quiera utilizar esta función, se debe llamar en el **void loop()** o en el **void setup()** de la siguiente manera, preocupándose de pasarle los datos necesarios:

```
1 void loop(){
2     mi_funcion_input(1,"b");
3 }
```

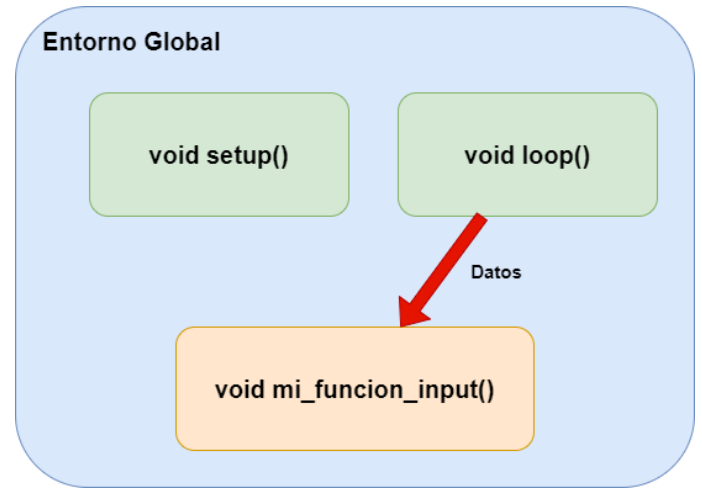


Figura 14: Diagrama de una función con input.

### 2.8.3. Funciones con salida(Output)

Ahora tenemos el caso contrario, en el que queramos obtener la información generada dentro de una función, esto se denomina como el retorno de la función, y para lograr esto, se tiene que especificar el tipo de dato que se quiere devolver, quedando algo de la siguiente manera:

```
1 int mi_funcion_output(){
2     int variable = 1;
3     //aquí va el código de la acción que se
4     quiere hacer
5     return variable;
}
```

Notar que en vez de void, se escribe el tipo de variable que se quiere retornar. Consideren que no se pueden retornar más de una variable de esta forma. También es importante fijarse en cómo se utiliza dentro del **void loop()** o en el **void setup()**:

```
1 void loop(){
2     int variable;
3     variable = mi_funcion_output();
4 }
```

Fíjense que la función debe ser asignada a una variable del mismo tipo que retorna para guardar la información.

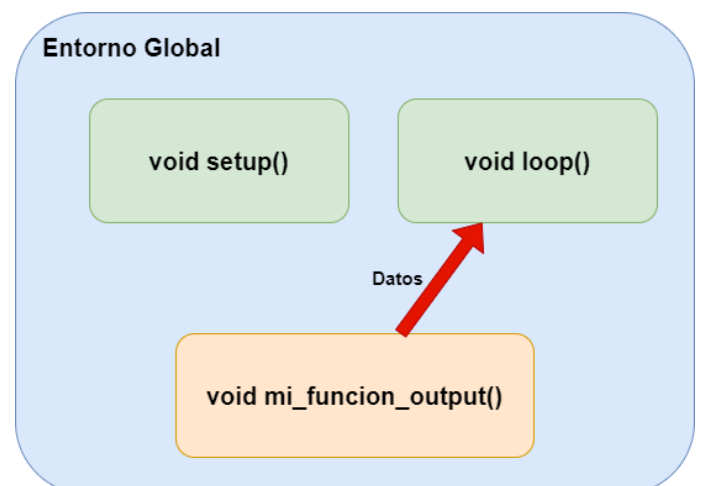


Figura 15: Diagrama de una función con output.

### 2.8.4. Funciones Input-Output

Por último, se pueden crear funciones que reciban información, las procesen, y retornen un resultado, lo cual se logra uniendo los dos métodos vistos anteriormente, por ejemplo, en esta función de suma:

```
1  int suma(int a,int b){
2      int resultado = a + b;
3      return resultado;
4  }
```

## 2.9. Desafíos

Determina el valor de verdad de las siguientes expresiones:

1. (Verdadero && Falso) || Verdadero
2. (false || false) || true
3. true && (false && false)

Determina el valor del elemento faltante para que toda la expresión sea verdadera:

1. (Verdadero && ???) && Verdadero
2. ??? || ((true && false) || false)

Determina el valor del elemento faltante para que toda la expresión sea falsa:

1. (??? || Falso) && Verdadero
2. ((true && false) || true) && ((true && ???) && true)

Determina qué parte del código se va a ejecutar:

```
1  float a = 3.14;
2  float b = 3.09;
3  bool c = false;
4  if(a < b){
5      //parte 1
6  }else if((a > b) && c){
7      //parte 2
8  }else{
9      //parte 3
10 }
```

```
1  void setup(){
2      float a = 2.5;
3      int z = funcion(a);
4      if (z==1){
5          //parte 1
6      }else if(z!=2){
7          //parte 2
8      }else{
9          //parte 3
10     }
11 }
12
13 int funcion(float x){
14     if(x < 1){
15         return 1
16     }else if (x > 2.49){
17         return 2
18     }else{
19         return 3
20     }
21 }
```

Determina cuántas veces se va a ejecutar cada código:

```
1  int i = 3;
2  while(i < 6){
3      //codigo
4      i = i - 1;
5  }
```

```
1  int i = 3;
2  while(i < 6){
3      for(int j=0; j<3;j++){
4          //codigo
5      }
6      i = i + 1;
7  }
```

Determina cuántas veces se va a ejecutar cada parte del código y en qué orden:

```
1  int f = 5;
2  for(int i = 0; i<6;i++){
3      if(i == f){
4          //parte 1
5      }else if(i<3){
6          //parte 2
7      }else{
8          //parte 3
9      }
10 }
```